

Object Design Document

Sistema per la gestione dell'identità digitale
degli studenti delle istituzioni AFAM

Insegnamento di Ingegneria del Software / Progettazione del Software

Anno Accademico 2025 – 2026

Documento di Analisi dei Requisiti

Gruppo:
Chiarelli Antonino Maria
Riccardo Centonze
Federico Meli
Gabriele Mattia Guccione

Indice

Indice	1
1. Introduzione	1
1.1 Prestazioni vs costi	1
1.2 Interfacce	1
1.3 Tempi di risposta	1
1.4 Linee guida per la documentazione delle interfacce	1
1.5 Linee guida per la documentazione del DBMS	1
2. Packages	1
2.1 com.afam.identity (backend)	1
2.1.1 entity	1
2.1.2 boundary	1
2.1.3 control	1
2.1.4 service	1
2.1.5 config	1
2.1.6 dto	1
2.2 Client Angular (frontend)	1
2.3 Librerie e dipendenze	1
3. Object Design	1
3.1 Classi entità	1
3.1.1 UtenteAfam	1
3.1.2 Profilo	1
3.1.3 Contenuto	1
3.1.4 Allegato	1
3.1.5 Gruppo	1
3.1.6 Link	1
3.1.7 Token	1
3.2 Oggetti control e boundary	1
3.2.1 Gestione Account (autenticazione e registrazione)	1
3.2.2 Gestione Profilo	1
3.2.3 Caricamento e Gestione Contenuti	1
3.2.4 Condivisione	1
3.2.5 Ricerca e Visualizzazione	1
3.2.6 Gestione errori e messaggistica	1
3.3 Strato di persistenza (DBMSBoundary)	1
3.4 Vincoli, eccezioni e visibilità	1

1. Introduzione

Il presente Object Design Document (ODD) raffina i modelli di analisi del RAD e di sistema dell'SDD per il sistema di gestione dell'identità digitale degli studenti AFAM, portandoli a livello di dettaglio implementativo. Coerentemente con le attività di object design, per ogni classe si specificano attributi e operazioni mancanti, i tipi, le signature e la visibilità, i vincoli e le eccezioni, e si esplicitano le librerie di classi riutilizzate. Il sistema è realizzato come applicazione web a tre livelli: un client a singola pagina (Single Page Application) in Angular, un'applicazione server REST in Spring Boot e una base di dati PostgreSQL. L'organizzazione del codice segue il pattern architetturale Boundary-Control-Entity (BCE), con uno strato di persistenza (DBMSBoundary) realizzato tramite repository.

1.1 Prestazioni vs costi

Per contenere tempi e costi di sviluppo sono state adottate esclusivamente tecnologie open source e ampiamente supportate: Spring Boot e Spring Data JPA/Hibernate lato server, Angular e Bootstrap lato client, PostgreSQL come DBMS. Tale scelta consente prestazioni adeguate ai requisiti, riuso di componenti maturi e assenza di costi di licenza, semplificando inoltre la manutenzione e l'onboarding di nuovi sviluppatori.

1.2 Interfacce

L'interazione con l'utente è affidata a un'interfaccia web responsiva basata su componenti Angular, progettata per essere chiara e intuitiva in considerazione di un'utenza non necessariamente esperta in ambito informatico (requisito di usabilità del RAD). La comunicazione fra client e server avviene tramite interfacce REST su HTTP, con scambio di messaggi in formato JSON; lo stato di autenticazione è veicolato da un token JWT trasmesso nell'header Authorization.

1.3 Linee guida per la documentazione delle interfacce

Le interfacce fra sottosistemi sono documentate secondo il pattern BCE. Gli oggetti boundary lato client (componenti Angular) raccolgono l'input ed espongono i dati; gli oggetti control lato server (i RestController) implementano la logica applicativa ed espongono gli endpoint REST; le classi entity modellano i concetti del dominio. Per ogni operazione pubblica si specifica la signature completa (nome, parametri tipizzati, tipo di ritorno) e la relativa visibilità, secondo le convenzioni UML (+ pubblico, - privato, # protetto, ~ package).

1.4 Linee guida per la documentazione del DBMS

La persistenza è gestita con PostgreSQL. Il dialogo fra l'applicazione e la base di dati è mediato dallo strato DBMSBoundary, costituito da interfacce repository di Spring Data JPA (estensioni di JpaRepository) che traducono le operazioni del dominio in query SQL tramite il provider Hibernate. Le password sono memorizzate cifrate mediante l'algoritmo di hashing Argon2, in conformità ai requisiti di sicurezza del RAD. Lo schema fisico completo delle tabelle è documentato nell'SDD (Gestione dei dati persistenti).

2. Packages

Il software è organizzato in due moduli principali — l'applicazione server (`com.afam.identity`) e il client Angular — e nelle relative librerie di terze parti. Di seguito se ne descrive la suddivisione in package/cartelle.

2.1 `com.afam.identity` (backend)

Package principale dell'applicazione server, suddiviso nei seguenti sotto-package coerenti con il pattern BCE.

2.1.1 `entity`

Contiene le classi che modellano le entità del dominio di cui è necessaria la persistenza: `UtenteAfam`, `Profilo`, `Contenuto`, `Allegato`, `Gruppo`, `Link`, `Token`. Sono annotate come entità JPA e mappate sulle tabelle del DBMS.

2.1.2 `boundary`

Contiene lo strato di persistenza (`DBMSBoundary`): le interfacce repository `UtenteAfamDBMSBoundary`, `ProfiloDBMSBoundary`, `ContenutoDBMSBoundary`, `AllegatoDBMSBoundary`, `GruppoDBMSBoundary`, `LinkDBMSBoundary`, `TokenDBMSBoundary`, che estendono `JpaRepository` e incapsulano l'accesso ai dati.

2.1.3 `control`

Contiene gli oggetti control, realizzati come `RestController`, che implementano la logica applicativa ed espongono gli endpoint REST: `AuthControl`, `RegistrationControl`, `ProfileControl`, `ContentControl`, `UploadControl`, `ViewControl`, `ShareControl`, `SearchControl`, oltre al validatore `CodiceFiscaleValidator`.

2.1.4 `service`

Contiene i servizi applicativi trasversali: `JwtService` (generazione e verifica dei token JWT, inclusi i token ospite) ed `EmailService` (generazione OTP e invio email per 2FA e recupero credenziali).

2.1.5 `config`

Contiene la configurazione di sicurezza (`SecurityConfig`) e il filtro di autenticazione (`JwtAuthenticationFilter`): definisce l'encoder Argon2, la policy di sessione stateless, le regole di autorizzazione per i ruoli AFAM e GUEST e la configurazione CORS.

2.1.6 `dto`

Contiene gli oggetti di trasferimento dati scambiati con il client (`LoginRequest`, `AuthResponse`, `RegistrationRequest`, `OtpRequest`, `AnagraficaDTO`, `CredentialsDTO`, `PasswordDTO`), che disaccoppiano la rappresentazione di rete dalle entità persistenti.

2.2 Client Angular (frontend)

Modulo client a singola pagina. Le cartelle principali sono: `boundary`, contenente i componenti di interfaccia (login, registrazione, otp, profilo e relative modifiche, gestione e caricamento contenuti, gruppi, condivisione e link, ricerca e risultati, visualizzazione pubblica di profili e contenuti, pagine di messaggio ed errore); `services`, contenente i servizi di accesso alle API REST (es. `AuthService`, `ProfileService`, `ContentService`, `GroupService`, `LinkService`); `guards`,

contenente AuthGuard per la protezione delle rotte riservate; e la configurazione di routing (app.routes.ts).

2.3 Librerie e dipendenze

Il sistema riusa le seguenti librerie di terze parti.

Libreria	Ruolo nel sistema
Spring Boot	Framework dell'applicazione server e gestione del ciclo di vita dei componenti.
Spring Web (REST)	Esposizione degli endpoint REST tramite RestController.
Spring Security + JWT	Autenticazione/autorizzazione stateless, filtro JWT, ruoli AFAM/GUEST.
Argon2 (Spring Security Crypto)	Hashing sicuro delle password (requisito ACN del RAD).
Spring Data JPA / Hibernate	Strato di persistenza e mapping oggetto-relazionale verso PostgreSQL.
Driver JDBC PostgreSQL	Connettività con il DBMS.
Angular	Framework del client a singola pagina.
Bootstrap	Stili e layout responsivo dell'interfaccia.
Libreria email (SMTP)	Invio di OTP ed email di recupero credenziali.

3. Object Design

Questa sezione raffina il modello degli oggetti del RAD specificando, per ciascuna classe, attributi e operazioni con tipo, signature e visibilità, secondo le attività di object design. Le classi entità sono presentate per prime; seguono, per macro-area, gli oggetti control (RestController) e i relativi boundary, secondo il pattern BCE. La notazione di visibilità è quella UML: + pubblico, - privato, # protetto, ~ package.

3.1 Classi entità

3.1.1 UtenteAfam

Modella l'account di autenticazione dell'Utente AFAM. È in relazione uno-a-uno con Profilo e uno-a-molti con Token.

Attributo	Tipo	Visibilità
uuid	String	-
username	String	-
email	String	-
password	String (hash Argon2)	-
has2FA	boolean	-
metodo2FA	String	-
registrationWithIdentityProvider	boolean	-
identityProvider	String	-

Operazioni (signature): + getUserId(): String, + getUsername(): String, + setUsername(username: String): void, + getEmail(): String, + setEmail(email: String): void, + getPassword(): String, + setPassword(password: String): void, + getHas2fa(): boolean, + setHas2fa(value: boolean): void, oltre alle operazioni di dominio individuate nel RAD (registration, login, logout, validateCredential, generateAccessToken, recoveryPassword, recoveryEmail), realizzate dagli oggetti control.

3.1.2 Profilo

Rappresenta l'identità rappresentativa dello studente; è la risorsa soggetta a policy di visibilità e a condivisione. Aggrega i value object Anagrafica e DatiAccademici (mappati come campi). In relazione uno-a-molti con Contenuto e Gruppo.

Attributo	Tipo	Visibilità
id	Long	-
utenteAfam	UtenteAfam (1:1)	-
descrizione	String	-
interessi	List<String>	-
competenze	List<String>	-
policyVisibilita	String {Tutti/Privato/categoria}	-
nome	String	-
cognome	String	-

dataNascita	LocalDate	-
codiceFiscale	String	-
citta	String	-
indirizzo	String	-
telefono	String	-
istituzione	String	-
dominioIstituzionale	String	-
matricola	String	-
corsoDiStudi	String	-
annoAccademico	String	-

Operazioni (signature): + getProfilo(): Profilo, + modifyAnagrafica(a: AnagraficaDTO): void, + modifyDatiAccademici(d: DatiAccademiciDTO): void, + modifyAltreInformazioni(): void, + aggiornaVisibilitaProfilo(policy: String): void, + calculateCF(): String, + validateBirthday(): boolean.

3.1.3 Contenuto

Modella un contenuto multimediale del portfolio, eventualmente composto da più allegati. In relazione multi-a-molti con Gruppo e uno-a-molti con Link.

Attributo	Tipo	Visibilità
id	Long	-
titolo	String	-
tipo	String {audio/video/immagine/elaborato}	-
descrizione	String	-
policyVisibilita	String {Tutti/Privato/Nessuno/categoria}	-
numeroVisualizzazioni	Integer	-
profilo	Profilo (proprietario)	-
allegati	List<Allegato>	-
autori	List<String>	-
collaboratori	List<String>	-
linkAssociati	List<Link>	-

Operazioni (signature): + caricaContenuto(): void, + modificaContenuto(): void, + eliminaContenuto(): void, + aggiungiAutore(nome: String): void, + aggiungiCollaboratore(nome: String): void, + rimuoviAllegato(a: Allegato): void, + aggiornaVisibilita(policy: String): void, + getAnteprima(): byte[], + checkFormato(): boolean, + getNumeroVisualizzazioni(): int, + getLink(): List<Link>, + setLink(l: Link): void.

3.1.4 Allegato

Reifica l'attributo multivalore «allegati» del Contenuto: ogni allegato corrisponde a un file fisico associato a un contenuto.

Attributo	Tipo	Visibilità
id	Long	–
urlFile	String (percorso del file)	–
contenuto	Contenuto (N:1)	–

3.1.5 Gruppo

Insieme astratto di due o più contenuti di un profilo, condivisibile con un unico link senza alterare l'organizzazione scelta dall'utente.

Attributo	Tipo	Visibilità
id	Long	–
nome	String	–
profilo	Profilo (proprietario)	–
listaContenuti	List<Contenuto> (N:M)	–
linkAssociati	List<Link>	–

Operazioni (signature): + creaGruppo(): void, + aggiungiContenuto(c: Contenuto): void, + rimuoviContenuto(c: Contenuto): void, + groupExist(): boolean, + getContenuti(): List<Contenuto>, + eliminaGruppo(): void, + getLink(): List<Link>, + setLink(l: Link): void.

3.1.6 Link

Collegamento univoco di condivisione verso una risorsa (Profilo, Contenuto o Gruppo), con stato e data di scadenza. Le tre associazioni sono opzionali: ne è valorizzata una sola in base alla risorsa condivisa.

Attributo	Tipo	Visibilità
identificatoreLink	String (PK)	–
dataScadenza	LocalDateTime	–
risorsaAssociata	String {Profilo/Contenuto/Gruppo}	–
stato	String {ATTIVO/REVOCATO}	–
impostazioni	String	–
numeroVisualizzazioni	Integer	–
profilo	FK (opzionale)	–
gruppo	FK (opzionale)	–
contenuto	FK (opzionale)	–

Operazioni (signature): + generaLink(): void, + disabilitaLink(): void, + modificaImpostazioni(): void, + checkLink(): boolean, + isAttivo(): boolean, + incrementaVisualizzazioni(): void.

3.1.7 Token

Stringa temporanea generata dal sistema per la conferma dell'identità (registrazione/2FA) o per il recupero delle credenziali. In relazione multi-a-uno con UtenteAfam.

Attributo	Tipo	Visibilità
id	Long	–
valore	String	–
scadenza	LocalDateTime	–
tipo	String {registrazione/OTP_2FA/recuperoPassword}	–
utenteAfam	UtenteAfam (N:1)	–

Operazioni (signature): + generaToken(): String, + checkToken(valore: String): boolean, + isScaduto(): boolean, + inviaViaMail(destinatario: String): void.

3.2 Oggetti control e boundary

Per ciascuna macro-area si riportano l'oggetto control (RestController lato server) con le principali operazioni esposte e i corrispondenti oggetti boundary (componenti Angular lato client). Lo strato di persistenza è condiviso ed è realizzato dalle interfacce DBMSBoundary.

3.2.1 Gestione Account (autenticazione e registrazione)

Control: AuthControl, RegistrationControl. Servizi di supporto: JwtService, EmailService; validazione CodiceFiscaleValidator.

Operazione (control)	Signature
AuthControl.login	+ login(request: LoginRequest): ResponseEntity
AuthControl.verifyOtp	+ verifyOtp(request: OtpRequest): ResponseEntity
AuthControl.guestLogin	+ guestLogin(): ResponseEntity
AuthControl.setup2fa / confirm2fa	+ setup2fa(req: HttpServletRequest): ResponseEntity / + confirm2fa(otp: OtpRequest, req): ResponseEntity
RegistrationControl.register	+ register(request: RegistrationRequest): ResponseEntity
JwtService.generateToken	+ generateToken(claims: Map, user: UserDetails): String
EmailService.sendOtpEmail	+ generateOtp(): String / + sendOtpEmail(to: String, otp: String): void

Boundary (Angular): LoginBoundary, RegistrationBoundary, OtpBoundary, IdentityProviderBoundary, RecoveryBoundary, RecoveryResetBoundary.

3.2.2 Gestione Profilo

Control: ProfileControl. Espone la lettura e l'aggiornamento del profilo dell'utente autenticato e il profilo pubblico.

Operazione (control)	Signature
getMe	+ getMe(): ResponseEntity<Profilo>
updateAnagrafica	+ updateAnagrafica(request: AnagraficaDTO): ResponseEntity
updateCredentials	+ updateCredentials(request: CredentialsDTO): ResponseEntity
updatePassword	+ updatePassword(request: PasswordDTO): ResponseEntity
getPublicProfile	+ getPublicProfile(id: Long): ResponseEntity

Boundary (Angular): ProfileBoundary, EditAnagraficaBoundary, EditCredentialsBoundary, EditPasswordBoundary, OtherEditsBoundary, PublicProfileBoundary.

3.2.3 Caricamento e Gestione Contenuti

Control: UploadControl (caricamento e creazione contenuto+allegato), ContentControl (gestione contenuti e gruppi), ViewControl (dettaglio e download).

Operazione (control)	Signature
UploadControl.upload	+ upload(file: MultipartFile, titolo, descrizione, policyVisibilita, autori, collaboratori): ResponseEntity
ContentControl.getContents	+ getContents(): ResponseEntity<List<Contenuto>>
ContentControl.deleteContent	+ eliminaContenuto(id: Long): ResponseEntity
ContentControl.aggregaContenuto	+ aggregaContenuto(gruppoId: Long, contenutoId: Long): ResponseEntity
ViewControl.getDettaglioContenuto	+ getDettaglioContenuto(id: Long): ResponseEntity
ViewControl.downloadFile	+ downloadFile(allegatoId: Long): ResponseEntity

Boundary (Angular): ContentManagementBoundary, UploadContentBoundary, ContentViewBoundary, PreviewBoundary, GroupManagementBoundary, GroupViewBoundary, PublicContentViewBoundary.

3.2.4 Condivisione

Control: ShareControl. Gestisce la generazione, revoca e fruizione pubblica dei link di condivisione.

Operazione (control)	Signature
createLink	+ generaLink(contenutoId: Long, giorni: int): ResponseEntity<Link>
revocaLink	+ disabilitaLink(identificatore: String): ResponseEntity
visualizzaCondivisione	+ visualizzaCondivisione(identificatore: String): ResponseEntity
downloadCondivisione	+ downloadCondivisione(identificatore: String): ResponseEntity

Boundary (Angular): ShareBoundary, ActiveLinksBoundary, LinkSettingsBoundary.

3.2.5 Ricerca e Visualizzazione

Control: SearchControl (ricerca globale su profili, contenuti pubblici e gruppi), ViewControl (visualizzazione pubblica).

Operazione (control)	Signature
SearchControl.ricercaGlobale	+ ricercaGlobale(q: String): ResponseEntity
ViewControl.getPublicContenuto	+ getPublicContenuto(id: Long): ResponseEntity

Boundary (Angular): ResultsBoundary, HomeBoundary.

3.2.6 Gestione errori e messaggistica

Coerentemente con i casi d'uso di comportamento eccezionale del RAD, le condizioni di errore (caduta connessione, errori di ricerca/condivisione/apertura link, errori di modifica profilo) sono gestite lato client da boundary di messaggistica transitoria — MessageBoundary, ErrorPageBoundary, ErrorMessageBoundary — e lato server dai codici di stato HTTP restituiti

dai control (es. 401 Unauthorized, 404 Not Found, 410 Gone per link scaduti/revocati). Queste boundary sono specializzazioni semantiche del medesimo concetto di notifica all'utente.

3.3 Strato di persistenza (DBMSBoundary)

Lo strato di persistenza è realizzato da interfacce repository che estendono JpaRepository, esponendo sia le operazioni CRUD standard sia query derivate dal nome del metodo. Esempi di signature:

Repository	Operazioni principali
UtenteAfamDBMSBoundary	findByUsername(username): Optional<UtenteAfam>; findByEmail(email): Optional<UtenteAfam>
ProfiloDBMSBoundary	findByUtenteAfamUuid(uuid): Optional<Profilo>; findByNome...OrCognome...(n, c): List<Profilo>
ContenutoDBMSBoundary	findByProfilo(p): List; findByTitoloContainingIgnoreCaseAndPolicyVisibilita(t, v): List
GruppoDBMSBoundary	findByNomeContainingIgnoreCase(n): List; findByProfiloId(id): List
LinkDBMSBoundary	findById(identificatore): Optional<Link>; findByProfiloId(id): List
TokenDBMSBoundary	findByValore(v): Optional<Token>; findByUtenteAfamAndTipoAndValore(u, t, v): List
AllegatoDBMSBoundary	findById(id): Optional<Allegato>; save / delete (CRUD)

3.4 Vincoli, eccezioni e visibilità

Conformemente alle attività di object design, oltre a tipi e signature si specificano i principali vincoli ed eccezioni.

Vincoli: la password deve contenere almeno 12 caratteri ed è memorizzata con hash Argon2; l'email deve rispettare l'espressione regolare di validazione; gli input testuali sono sanificati e non possono contenere i caratteri riservati indicati nel RAD; il codice fiscale è validato e ricalcolato dai dati anagrafici; ogni identificatore di link è univoco e dotato di data di scadenza. Eccezioni e condizioni d'errore sono propagate dai control come risposte HTTP con codice e messaggio dedicati (autorizzazione negata, risorsa non trovata, link scaduto o revocato, errore di persistenza), gestite dal client con redirect alle pagine di errore.

Visibilità: gli attributi delle entità sono privati e accessibili tramite metodi di get/set pubblici; le operazioni esposte dai control sono pubbliche e costituiscono l'interfaccia REST del sistema.